Claude Code on Lubuntu: Use and Best Practices

Oct 27, 2026 · Joe Solch

What you get

One Lubuntu 26.04 machine per person, with Claude Code running as one normal user and never as root. Everything is installed straight onto the OS: no Docker, no LXC and no other containers. Claude Desktop runs on the machine's own desktop with Chat, Code and Cowork. The machine can be a real PC or a Proxmox VM (see "Running in a Proxmox VM" below).

Part	Where it lives	Owned by
Claude Code	~/.local/bin/claude	the user
Claude settings, plugins, skills	~/.claude/	the user
Projects	/project	the user
Claude Desktop (Chat, Code, Cowork)	`claude-desktop`, from Anthropic's apt repository	root; runs as the user on the desktop
Cowork's VM stack	QEMU, OVMF firmware, virtiofsd; `kvm` group; `vhost_vsock` module	root; the user is in the `kvm` group
Node.js, Python, Go, language servers	/usr, /usr/local	root
Rust toolchain	/opt/rust (your crates in ~/.cargo)	root
GitHub CLI (`gh`)	/usr/bin/gh, from GitHub's apt repository	root; you sign in as the user
Playwright browsers	/opt/ms-playwright	root; everyone can read them
Web UI (CloudCLI)	port 3001, service `cloudcli`	runs as the user
Update and health tools	`agentic-update`, `agentic-doctor`, `agentic-setup-user`, `agentic-errorlog`	root (run with sudo)
Problem log	Agentic-Errors.txt on the user's desktop, only when something fails	the user
Phone alerts (optional)	ntfy, set in /etc/agentic/ntfy.conf	root
Notes card	/etc/agentic/README.md (copy in ~/CLAUDE-BOX.md)	root

Before you run the script

1. **Install Lubuntu 26.04** and create your own account in the installer. That account is the natural choice for the Claude user, because it's the one the desktop logs in as.
2. **Turn on hardware virtualization** in the BIOS/UEFI setup (Intel VT-x or AMD-V, sometimes called "SVM Mode"). Cowork needs it. On a Proxmox VM, see the next section instead.
3. **Run the script** from a terminal on the machine: `sudo bash Claude_Code_Lubuntu_Deployer.sh`. It asks all its questions first and changes nothing until you answer "Proceed? y".

The script checks for hardware virtualization before it starts. If it's missing, you get a warning and the install continues: Chat and Code work, and Cowork starts working as soon as virtualization is turned on. You don't need to run the script again.

Running in a Proxmox VM

The script runs unchanged in a Proxmox Lubuntu VM, because to the script a VM looks like a normal Lubuntu machine. What needs changing is the VM's settings in Proxmox.

- **Cowork needs nested virtualization.** Cowork runs its own small VM, so this VM has to pass virtualization through. Set the VM's CPU type to `host` (Hardware → Processors → Type, or `qm set <vmid> --cpu host`), then stop and start the VM; a reboot from inside it isn't enough.
- **Nested KVM must be on on the Proxmox host.** On the host, `cat /sys/module/kvm_intel/parameters/nested` (or `kvm_amd` on AMD) should print `Y` or `1`. It's on by default on current Proxmox kernels. If it isn't, put `options kvm-intel nested=Y` (or `options kvm-amd nested=1`) in `/etc/modprobe.d/kvm.conf` on the host and reboot the host.
- **Without nested virtualization** the script still installs everything, but warns that Cowork can't run, because the VM has no `/dev/kvm`. Fix the two settings above and Cowork works after the VM's next start.
- **Memory and disk.** Give the VM at least 8 GB of RAM, 4 or more cores and about 40 GB of disk. Claude Desktop, Cowork's own VM and the dev tools all share it.
- **Live migration.** CPU type `host` ties the VM to the host's CPU model, so it can't live-migrate to a node with a different CPU. Snapshots and backups work as usual.
- **Nice to add (optional, not done by the script).** Inside the VM, run `sudo apt install qemu-guest-agent`, then turn on Options → QEMU Guest Agent in Proxmox and restart the VM. Proxmox can then shut the VM down cleanly and show its IP address.

To check from inside the VM after the install, run `sudo agentic-doctor`. The "hardware virtualization on (/dev/kvm)" line should say OK.

First steps after deploy

Do these right after the script finishes, in this order; step 1 matters most.

1. **Claim the web UI.** It listens only on this machine unless you chose LAN at deploy. From your PC, run `ssh -L 3001:localhost:3001 <user>@<ip>` and open `http://localhost:3001` (with LAN access, open `http://<ip>:3001` instead). Create its login straight away: until you do, anyone who can reach it can create it.
2. **Log out and back in, or reboot.** The script offers a reboot at the end. The `kvm` group that Cowork needs only applies to new logins.
3. **Sign in to Claude Desktop.** Log in to the desktop as your user, open **Claude** from the menu (or run `claude-desktop`) and sign in. Try the Cowork tab; if it reports a missing requirement, run `sudo agentic-doctor`.
4. **Sign in to Claude Code.** In a terminal, run `claude` and follow `/login`. A claude.ai subscription also enables Remote Control.
5. **Check health.** Run `sudo agentic-doctor`. Every line should say OK. The expected WARN is "logged in", and only before step 4. Any WARN or FAIL is also copied to Agentic-Errors.txt on your desktop, so if you ran it early, delete that file.
6. **Set your git identity.** Run `git config --global user.name "..."` and `git config --global user.email "..."`.
7. **Sign in to GitHub.** Run `gh auth login` once (choose HTTPS and let it set up git credentials). Claude uses `gh` for pull requests and issues.
8. **Keep the install log.** It's under `/var/log/agentic-deploy/`, with the provisioning trace in `/var/log/agentic-provision.log`.

Daily use

Log in as your user, `cd` into a project under /project, and run `claude` there. Claude reads that project's files and the shared /project/CLAUDE.md.

- **Start a project:** `mkdir -p /project/src/myapp && cd $_ && git init`, then `claude`.
- **Resume:** `claude --continue` picks up the last session in that folder; `claude --resume` lets you choose one.
- **Claude Desktop:** the Code tab works on /project directly; Cowork handles longer agentic tasks in its own VM, which the app starts and stops by itself. Don't start, stop or edit its QEMU processes yourself.
- **Web UI:** chat, a file editor, git and a shell, all running as your user. By default it's reachable only through an SSH tunnel: `ssh -L 3001:localhost:3001 <user>@<ip>`, then `http://localhost:3001`. Re-run the deploy script and answer `lan` to open it on `http://<ip>:3001`. To open it to your LAN later, change `HOST=127.0.0.1` to `HOST=0.0.0.0` in /etc/systemd/system/cloudcli.service, then run `sudo systemctl daemon-reload && sudo systemctl restart cloudcli`.
- **From another PC:** Claude Desktop on your own computer can open an SSH session to this machine (`ssh-copy-id <user>@<ip>` first; Desktop can't type passwords).

- **From your phone:** in a session, type `/rc` (Remote Control), then continue from the Claude app or claude.ai/code.
- **No Docker:** there are no containers here. If a project needs a database server, run `sudo apt install postgresql` (or `redis-server`) yourself and use it as a normal service.
- **Admin commands:** run `sudo apt install ...` and other root tasks yourself. Claude is blocked from sudo and su, so when it needs one it should ask you.
- **Python:** use a venv per project (`python3 -m venv .venv`) rather than system-wide installs.
- **Node:** `npm i -g` installs into your own ~/.local, so no sudo is needed. npm skips install scripts for packages you haven't approved and lists them after the install; approve one you trust with `npm approve-scripts` in that project, or `--allow-scripts=<pkg>` on a global install.

Maintenance

Updates run on their own every Sunday at 4 AM local time; you only step in when the health check fails.

When an update or health check hits a WARN or FAIL, an entry is appended to `Agentic-Errors.txt` on your desktop (your home folder if there is no desktop). Each entry has the time, the step that failed and the last 40 lines of the log. If the file is there, something needs a look; if it isn't, every run since you last deleted it was clean. Entries older than 90 days are trimmed and the file is capped at 256 KB. Delete it once you've dealt with the problems.

Task	Command	Notes
Update everything now	`sudo agentic-update`	OS (incl. Claude Desktop and QEMU), npm and the global dev tools (including Playwright and gopls), Rust, Claude Code and the web UI, then a health check
Health check	`sudo agentic-doctor`	Exit 0 means healthy; includes Cowork's KVM, QEMU and vsock checks, and warns if sudo stops asking for a password
Check for problems	`cat ~/Desktop/Agentic-Errors.txt`	Only exists after a WARN or FAIL; delete it when dealt with
Test phone alerts	`sudo agentic-errorlog --test-ntfy`	Only if ntfy is set up; see the next section
Read the update log	`less /var/log/agentic-update.log`	Rotated monthly
Repair the Claude setup	`sudo agentic-setup-user`	Reinstalls Claude if it's missing or broken, re-adds the plugins and skills, and adds any new deny rules; the rest of your settings.json is kept
Web UI status and logs	`systemctl status cloudcli`, `journalctl -u cloudcli`	
Install logs	`/var/log/agentic-deploy/`, `/var/log/agentic-provision.log`	The provision log includes a trace of every command

A new kernel from an update only takes effect after a reboot; the update log says when one is needed. Back up /project and ~/.claude regularly. On a Proxmox VM, take a snapshot before big changes (`qm snapshot <vmid> before-change`).

Phone alerts with ntfy (optional)

Agentic-Errors.txt only helps when you're at the machine. If you also want a push on your phone, `agentic-errorlog` can send one through ntfy every time it writes an entry. It stays off until you set a topic. The push is short (how many problems, the first few lines, and which machine); the file stays the full record.

1. **Pick a topic name that's hard to guess.** On the public ntfy.sh server a topic has no password: anyone who knows its name can read your alerts and post fake ones. Use something long and random, such as `claude-alerts-` followed by the output of `openssl rand -hex 8`. Avoid your name, the hostname, or anything obvious like `claude-errors`.
2. **Subscribe on your phone.** Install the ntfy app (Android or iOS), tap **+**, and enter the same topic (and your server, if you use your own).
3. **Give the topic to the machine.** Answer the ntfy prompt when you run the deploy script (re-running it and pressing Enter keeps the current setting), or set it later with `sudo nano /etc/agentic/ntfy.conf`.
4. **Test it.** Run `sudo agentic-errorlog --test-ntfy`. A test push should arrive within seconds.

The settings live in `/etc/agentic/ntfy.conf`, which only root can read because it can hold a token:

Setting	Meaning
`NTFY_TOPIC`	Topic name, or a full URL such as `https://ntfy.example.com/my-topic`. Empty turns alerts off.
`NTFY_SERVER`	Server for a bare topic name; default `https://ntfy.sh`. Change it if you run your own ntfy server.
`NTFY_TOKEN`	Access token (`tk_...`) for a server or topic that requires a login; leave empty otherwise.

For more privacy, reserve the topic with an ntfy.sh account or run your own server, and use an access token. To stop alerts, empty `NTFY_TOPIC`.

Best practices with Claude

Treat Claude like a fast, capable colleague with shell access. Give it clear goals and a way to check its work, and review what it changes.

- **Use git for everything.** Commit before a big task, so a bad change is one `git reset` away. Read diffs before you push.
- **Plan first on big tasks.** Press Shift+Tab for plan mode, or use `/superpowers:brainstorm` and then `/superpowers:write-plan`.
- **Give it a way to verify.** Tests, a build command, or the webapp-testing skill for UIs let Claude check itself.

- **Keep secrets out of reach.** Put credentials in `.env` files or a `secrets/` folder; the deny rules stop Claude reading those, `*.pem`, everything in ~/.ssh, and the gh, git and npm credential files. Never paste a token into the chat.
- **Write down project rules.** Add a CLAUDE.md to each project (build, test and style commands); keep the shared /project/CLAUDE.md short.
- **Watch auto mode.** It approves most actions on its own, behind a safety classifier and the deny rules. For risky work (production data, real infrastructure), switch to default mode with Shift+Tab, or use `/permissions`.
- **Don't run Claude as root.** The `claude` command refuses to run as root here; keep it that way.
- **Mind the memory.** Cowork's VM, Claude Desktop and a big build share the same RAM; run a few agents at a time, not many, on an 8 GB machine.
- **Manage context.** Use `/clear` between unrelated tasks, `/compact` on long ones, and `/effort` to spend more or less thinking.
- **Review plugin and MCP additions.** Anything you install runs as your user; add only sources you trust.

Security model

There's no container around Claude here, so the user account is the boundary. Claude can do whatever your user can, and nothing that needs root without you typing the sudo password.

Layer	What the script does	Why
Claude user	Normal account; root is blocked from running `claude`	Limits what an agent mistake touches
Passwords	The user's password must differ from root's	A leaked user password doesn't become root
sudo	Optional, and asks for a password. The script checks every sudoers file for a NOPASSWD rule that covers the user (directly, through ALL, or through a group such as `sudo`) and offers to remove it; `agentic-doctor` warns if one comes back	Admin work stays with you; Claude can't type the password
Claude permissions	Auto mode, plus deny rules for secrets and credential files (~/.ssh, gh, git, npm), `dd`, `mkfs`, `rm -rf /`, `sudo` and `su`	Claude can't read credentials or escalate on its own
npm install scripts	Allowed only for the web UI's four native modules; the weekly tool updates run with none	A compromised package can't run code as root when it's installed
Cowork	Runs tasks in its own QEMU/KVM VM; the user is in the `kvm` group	The VM is the app's sandbox; `kvm` grants VM access, not root
SSH	The user logs in with a password or key. Root is key only unless you choose "password" at deploy, which the script warns against (each re-run asks again); at most 4 tries per connection	A guessable root password over the network would be the whole machine; keep port 22 on your LAN

Layer	What the script does	Why
Web UI	Runs as the user; own login; listens on the machine only unless you chose LAN at deploy, and starts only after setup is finished	Plain HTTP on port 3001: use an SSH tunnel, or keep LAN access to a trusted network or behind a TLS reverse proxy
Phone alerts	Off unless you set an ntfy topic; settings file is root-only	A guessable topic would let others read your alerts
Updates	Weekly, with health check	Keeps OS, Claude Desktop and Claude Code patched

Further hardening you can add:

- Turn on the firewall (`sudo ufw allow from <your-lan> to any port 22 proto tcp`, plus port 3001 if you chose LAN access for the web UI, then `sudo ufw enable`), or the Proxmox firewall for a VM.
- Use SSH keys, and set PasswordAuthentication no in /etc/ssh/sshd_config.d/00-agentic.conf once they work.
- If you never need root over SSH, set PermitRootLogin no in that same file. (Re-running the script resets it to your key-only or password choice.)

Uninstalling

Run `sudo bash Claude_Code_Lubuntu_Deployer.sh --uninstall`. It asks its questions first, shows what it will do, and changes nothing until you type `uninstall`. Everything is logged under `/var/log/agentic-deploy/`.

Part	What happens
Web UI, the agentic-* tools, the weekly update, /etc/agentic, /usr/local/share/agentic, the root `claude` stub, the SSH and apt drop-ins, the shell block in .bashrc, the npm install-script setting	Always removed. SSH goes back to Ubuntu's defaults: root by key only, passwords allowed for everyone else
Go, Rust, gopls, rust-analyzer, the global npm tools, Playwright and its browsers	Asked; the default is yes
Node.js, gh, Claude Desktop, clangd, Cowork's QEMU/KVM stack and their apt repositories	Asked; the default is no, because other software may use them
The Claude user's Claude data (~/.claude, the claude install, ~/.cloudcli, the notes card, Agentic-Errors.txt)	Asked; the default is no. Deleting it signs Claude out and removes its settings, plugins and history
The user account, /project, root's password, hostname, timezone, the SSH server, base packages, group memberships and the sudoers edits	Kept. The sudoers backups stay in `/root/*.agentic-bak`

Log out and back in afterwards, so your shells drop the old PATH and settings.

Troubleshooting

Start with Agentic-Errors.txt on your desktop if it exists, then `sudo agentic-doctor`; each FAIL line names the part to look at. The comment block at the top of the deploy script has a "Where things live" list of every file it writes, and comments through the script explain each step.

Symptom	Check or fix
Deploy failed	Read `/var/log/agentic-provision.log`; the line before "provisioning FAILED" shows the failing command. Fix it and re-run the script; it's safe to run again.
Cowork: "requires hardware virtualization (KVM)"	On a PC, enable VT-x/AMD-V in the BIOS. On a Proxmox VM, set CPU type `host` and check nested KVM on the host (see "Running in a Proxmox VM"). `kvm-ok` explains what's missing.
Cowork: "doesn't have permission to use virtualization"	Log out and back in, or reboot, so the `kvm` group applies. Check with `id -nG`.
Cowork: "requires the vhost_vsock kernel module"	`sudo modprobe vhost_vsock`, then restart the app. The script already loads it on every boot.
Cowork: "requires QEMU"	`sudo apt install qemu-system-x86 ovmf virtiofsd`, then restart the app.
Claude Desktop forgets your sign-in	The desktop keyring is locked or missing: `sudo apt install gnome-keyring libpam-gnome-keyring`, then log out and in. If you changed your password in the script, the keyring may ask for the old one once.
`claude: not run as root`	Expected. Use `su - <user>`, then `claude`.
`claude: command not found` as the user	Run `sudo agentic-setup-user`.
Web UI won't open	It's local only unless you chose LAN: use the SSH tunnel above. If the tunnel doesn't work either, check `journalctl -u cloudcli -n 50`, then `sudo agentic-update` (it rebuilds native modules).
Doctor: "sudo asks for a password" WARN	A NOPASSWD rule (or `Defaults !authenticate`) lets the user use sudo without a password, which an auto-mode agent could use. `sudo -l -U <user>` shows it; remove it with `sudo visudo` or by editing the file in /etc/sudoers.d/, or re-run the deploy script and say yes when it offers to.
`sudo` asks for a password Claude can't give	Expected. Run the command yourself.
No ntfy push arrives	Run `sudo agentic-errorlog --test-ntfy`; it names the failure. Check the topic matches the one in the app, and the token if your server needs one.